Звіт

до лабараторної роботи №4
Об’єктно-орієнтоване програмування. Java
Модульне тестування за допомогою TestNG

Студент: Ісаєв Захар Ігорович
Група: КН-924В
Викладач: Володимир Миколайович Савченко
Рік: 2026

## Мета роботи

Набуття практичних навичок використання TestNG для модульного тестування Java-проєктів: побудови тестового Maven-проєкту, застосування анотацій життєвого циклу, групування тестів і параметризації через @DataProvider, налаштування запуску за допомогою testng.xml та порівняння підходів TestNG і JUnit Jupiter.

## 2. Опис предметної галузі

У даній лабораторній роботі розглядається система бронювання номерів у готелі. (Варіант 3)

Система бронювання дозволяє клієнтам резервувати номери на певну дату.

Система повинна зберігати інформацію про гостей, кімнати, готель та створені бронювання.

Основною функціональністю системи є:

- введення інформації про гостя
- введення інформації про кімнату
- створення бронювання
- збереження бронювань у готелі
- відображення інформації про об’єкти

Класи системи:

1. Guest — гість готелю
2. Room — кімната готелю
3. Hotel — готель
4. Reservation — бронювання
5. Main — клас запуску програми

## Модульне тестування

Тести проводилися за допомогою бібліотеки TestNG.

Тести розміщенні у пакеті: src\test\java\ua\khpi\oop\lab04

Тестування класу Guest:

Перевірено:

1. створення об’єкта Guest з коректними аргументами;
2. використання спільної фікстури @BeforeMethod, яка створює початковий об’єкт для тестів;
3. створення об’єкта лише з ім’ям, коли номер телефону автоматично встановлюється як "не вказано";
4. зміну стану об’єкта після виконання предметної операції changePhone().

Тестування класу Room:

1. створення об’єкта Room з коректними значеннями номера та ціни;
2. використання спільної фікстури @BeforeMethod, яка створює початковий об’єкт кімнати для тестів;
3. параметризовану перевірку методу isExpensive() за допомогою @DataProvider для різних значень ціни;
4. зміну стану об’єкта після виконання предметної операції changePrice();
5. негативний сценарій створення об’єкта з від’ємною ціною з перевіркою винятку IllegalArgumentException.

Тестування класу Hotel:

1. створення об’єкта Hotel з коректними аргументами;
2. використання спільної фікстури @BeforeMethod, яка створює початкові об’єкти готелю, гостя та кімнати;
3. додавання кімнати до списку кімнат готелю методом addRoom();
4. створення бронювання методом createReservation() та коректне додавання його до списку бронювань.

Тестування класу Reservation:

1. створення об’єкта Reservation з коректними аргументами;
2. використання спільної фікстури @BeforeMethod, яка створює початковий об’єкт бронювання для тестів;
3. зміну дати бронювання після виконання предметної операції changeDate();

4. коректність текстового представлення об’єкта, яке повертає метод toString().

Результати тестування

Усі модульні тести були успішно виконані.

Під час виконання тестів:

Помилок немає

Методи працюють коректно

Функціональність системи підтверджена

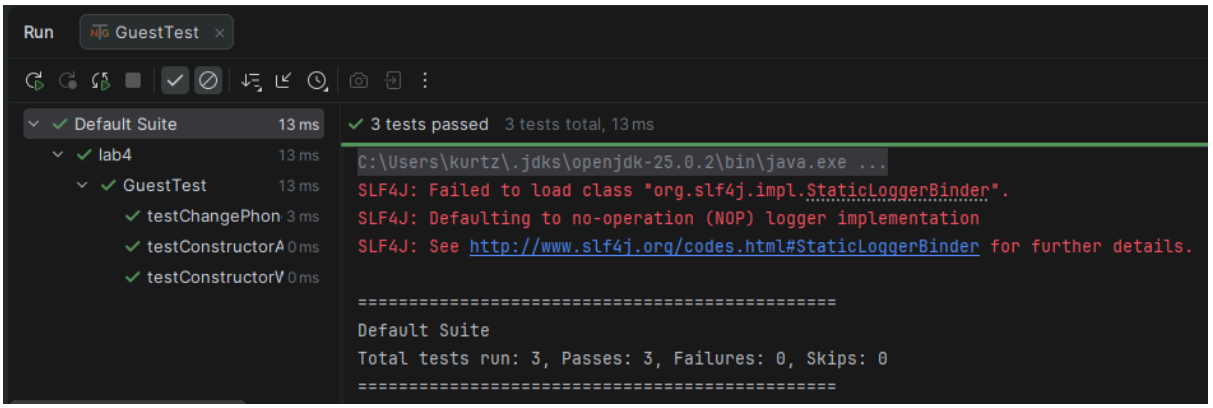


Рис. 1 - GuestTest

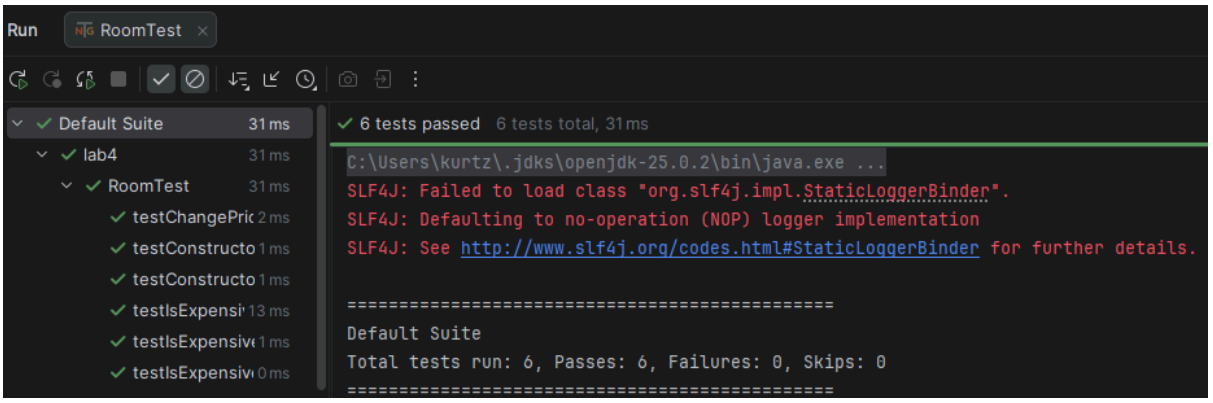


Рис. 2 - RoomTest

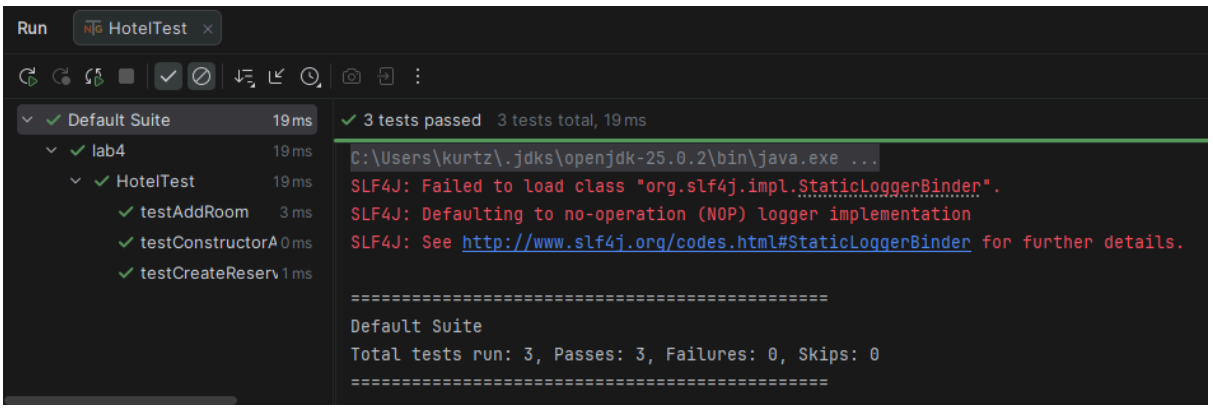


Рис. 3 - HotelTest

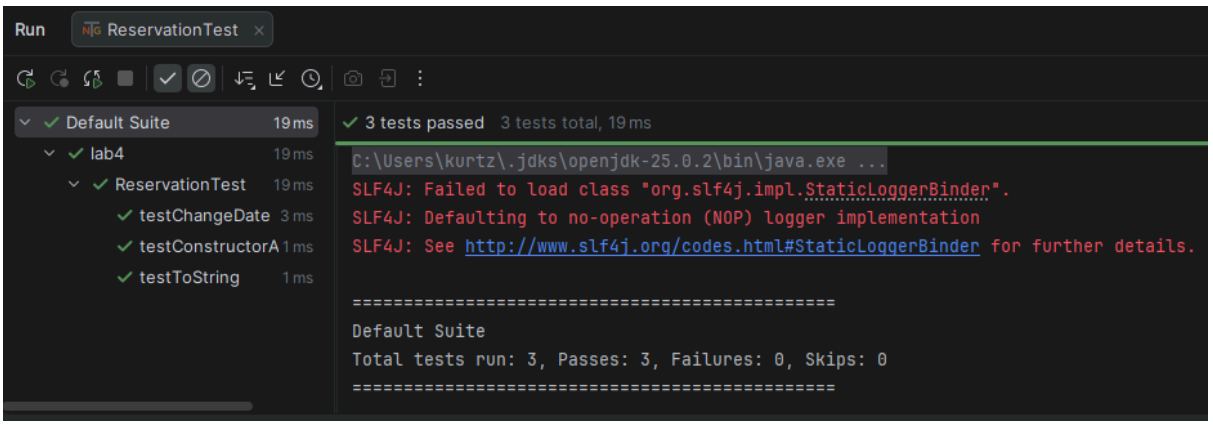


Рис. 4 - ReservationTest

Порівняння JUnit та TestNG

Було розглянуто відмінності між бібліотеками JUnit та TestNG

JUnit є однією з найпоширеніших бібліотек для модульного тестування. Вона відзначається зрозумілою структурою, простотою використання та широко застосовується під час розробки Java-застосунків.

TestNG, у свою чергу, має ширші можливості для організації тестування. Зокрема, ця бібліотека надає:

- підтримку пріоритетності виконання тестів;
- можливість об’єднання тестів у групи;
- підтримку залежностей між тестами;
- більш гнучкі засоби налаштування та конфігурації тестового процесу.

## Висновок

У ході виконання лабораторної роботи було створено тестові класи для перевірки основних компонентів моделі бронювання готелю. Під час роботи було використано можливості бібліотеки TestNG, зокрема фікстури життєвого циклу, групування тестів, параметризовані тести та перевірку винятків. У результаті було перевірено коректність створення об’єктів, зміну їхнього стану та роботу окремих предметних методів. Отже, використання TestNG дало змогу організувати тестування більш гнучко та зручно порівняно з базовими засобами модульного тестування.